BLOODGRID · CCP1

How the optimisation works, for blood donation

What the solver is actually deciding, and why it matters clinically

Companion to docs/CCP1_ALGORITHM_REVIEW.md and the implementation in ccp1/. Every figure in this document was produced by that code on fictional demonstration data.

> This explains the method in the language of a blood service rather than the language of operations research. The worked example uses the fourteen fictional donors from the prototype, and every number shown was computed, not invented.

1. The situation it solves

It is Tuesday morning at a regional blood centre. Three requests are open at the same time:

REQUEST	GROUP NEEDED	UNITS	URGENCY	SITUATION
REQ-A	O positive	2	Critical	Theatre, bleeding now
REQ-B	O positive	2	Urgent	Needed within hours
REQ-C	A positive	1	Routine	Scheduled transfusion

Fourteen donors are registered. Ten are eligible and compatible with at least one of these requests. **The three requests are competing for the same ten people.**

That competition is the whole problem. If you handle REQ-A first and give it the two best donors, those two are gone when you turn to REQ-B. The order you happen to process requests in changes who every patient gets. A method that looks at one request at a time cannot see this, because by the time it reaches the third patient it has already spent the pool.

The question the optimiser answers. Not "who is the best donor for this patient", but "given all three patients at once, which allocation of these ten people is best overall".

2. What the computer is actually deciding

One yes-or-no decision for every donor and request pairing. Ten donors and three requests gives thirty such decisions, laid out as a grid:

```
            REQ-A    REQ-B    REQ-C
            (O+)     (O+)     (A+)
DNR-001      1        0        0      Ahmed goes to patient A
DNR-002      1        0        0
DNR-003      0        0        0      Usman is not called today
DNR-007      0        1        0
...
```

In the model this is written `x[i][j] = 1` when donor *i* is assigned to request *j*, and 0 otherwise. Every possible way of staffing the day's requests is one filled-in grid. The solver's job is to find the best grid.

There are a great many grids. With ten donors and three requests the count runs into the millions, and a real service with hundreds of donors and dozens of open requests has more possibilities than could ever be listed one by one. That is why this needs a solver rather than a loop.

3. Rules that can never be broken

Some things are not trade-offs. They are clinical or physical facts, and the solver is forbidden from violating them:

RULE	WHY IT EXISTS	IN THE MODEL
A donor gives to one patient	A person donates one unit; they cannot be in two places	`sum over j of x[i][j] ≤ 1`
Every request gets enough donors	A patient needing two units needs at least two donors	`sum over i of x[i][j] ≥ k[j]`
Outreach per request is capped	You may contact a few spares in case some decline, but you must not message the entire register for one patient	`sum over i of x[i][j] ≤ cap[j]`
Blood groups must be compatible	Standard ABO and Rh matching. Non-negotiable	`x[i][j] = 0 for incompatible pairs`
Donors must have recovered	Ninety days between whole-blood donations	`x[i][j] = 0 for ineligible donors`

The cap rule was learned by running the model, not by thinking about it. The first version had only a lower bound on donors per request. The solver responded by assigning seven donors to a patient who needed one, because contacting one more person can only ever raise the chance that somebody says yes. Mathematically correct, and exactly the donor-spam behaviour a real service must avoid. The upper bound fixed it.

4. The four things being balanced

Within those hard rules, four goals pull in different directions. Each is a real concern for a blood service, not a mathematical convenience.

Will the patient actually get blood

Being assigned a donor is not the same as receiving blood. Donors decline, or do not answer. Each donor has a response probability: DNR-006 answers about 95 percent of the time, DNR-008 about 58 percent. Contacting more people, and better-responding people, raises the chance the patient is covered.

How fast can they get here

A patient is not helped by the average distance of their donors. They are helped when the *last* needed donor arrives. So what matters is the furthest donor assigned to a request, not the sum. This is why the model minimises a maximum, not a total.

How hard are we leaning on our donors

A donor who gave three months ago is technically eligible and still recovering. A donor called repeatedly stops answering. Burden measures strain: it is higher for donors close to their ninety-day boundary and for those already heavily used. Ignoring it burns out the register, which costs far more than one transfusion.

Are we wasting universal blood

This is the one that a simple scoring system cannot see at all. O negative blood can be given to any patient, so it is the most valuable stock a service holds. Spending an O negative donor on an O positive patient works clinically and is a quiet waste, because an O positive donor could have done the same job and the O negative unit may be the only thing that saves someone else later.

DONOR GROUP	PATIENTS IT CAN SERVE	HOW PRECIOUS
O negative	8 of 8	Universal donor, most valuable
O positive	4 of 8	Broadly useful
A negative	4 of 8	Broadly useful
A positive	2 of 8	Specific
AB positive	1 of 8	Universal recipient, least flexible as a donor

5. These four goals genuinely fight each other

This is not a rhetorical claim. Here are three real solutions the solver found for the same Tuesday morning, each optimal for a different goal:

OPTIMISED FOR	P(FULFIL) A / B / C	TOTAL DISTANCE	UNIVERSAL BLOOD WASTED
Best chance of fulfilment	0.994 / 1.000 / 0.995	37.4 km	12
Fastest response	0.987 / 0.857 / 0.860	6.5 km	4
No universal blood wasted	0.990 / 0.966 / 0.860	11.3 km	0

Read the first two rows. Chasing the highest chance of fulfilment means calling more distant donors and burning universal blood: distance rises from 6.5 km to 37.4 km. Chasing speed drops patient B's chance of being covered from essentially certain to 86 percent.

There is no single arrangement that wins on all four. That is what "multi-objective" means in practice, and it is why the honest output is a set of trade-offs for a human to choose from, rather than one answer presented as if it were the only one.

6. Why "just pick the best donor for each patient" fails

The prototype currently scores each donor for one request and takes the highest. Run that same logic across all three requests, most urgent first, and this is what it produces:

REQUEST	DONORS CHOSEN BY THE SIMPLE METHOD	COMMENT
REQ-A (O+)	DNR-001 (O+), DNR-002 (O+)	Sensible
REQ-B (O+)	**DNR-007 (O negative)**, DNR-009 (O+)	Universal donor used on an O+ patient
REQ-C (A+)	**DNR-003 (O negative)**	A second universal donor spent, on the least urgent patient

Two units of universal blood, gone. Meanwhile DNR-012, an A positive donor with a 0.86 response rate sitting 0 km from patient C, was never called.

The simple method is not being stupid. It is being *local*. When it reaches REQ-C it asks "who is the best remaining donor for this A positive patient", and an O negative donor genuinely does score well: compatible, reliable, available. Nothing in a per-request score can express "yes, but that person is the most valuable donor on the register and someone else can do this job."

> **What the optimiser does instead.** Give REQ-C its A positive donor, and both O negative donors stay on the shelf. Universal blood wasted: **zero** instead of ten. Patient C's chance of being covered falls from 0.88 to 0.86, a difference of two percentage points. Two units of universal blood preserved for two points of probability is a trade any blood service would take, and the simple method could not even see that the choice existed.

7. The trick that makes this solvable

The fulfilment goal looks like it should be hard. The chance that at least one of several donors says yes is:

```
P(patient covered) = 1 - (1 - p1) x (1 - p2) x (1 - p3) ...
```

That is a product, and products are difficult for solvers. Multiply things together and the fast, exact methods stop applying.

The fix is a piece of school mathematics. The logarithm of a product is the sum of the logarithms. So instead of multiplying the decline probabilities, add their logarithms:

DONOR	SAYS YES (P)	DECLINES (1 - P)	LOG(1 - P)
DNR-006	0.95	0.05	-2.996
DNR-001	0.92	0.08	-2.526
DNR-008	0.58	0.42	-0.868

A reliable donor gives a large negative number, an unreliable one a small negative number. Adding these up and pushing the total as low as possible selects exactly the same donors as maximising the original product would. Nothing is approximated. It is the same answer, in a form the solver can handle.

The other three goals are already sums, or can be written as one. So all four become linear: a coefficient multiplied by each yes-or-no decision, added together. **That single fact is what decided the method.** Linear problems can be solved exactly and quickly; the moment something stops being linear, exact solving is off the table.

8. How the solver actually searches

The solver does not test every grid. It uses branch and bound, which in donor terms works like this:

1. **Relax the yes-or-no rule.** Pretend a donor can be assigned "40 percent to patient A". Meaningless clinically, but this relaxed version can be solved instantly and exactly.
2. **If the answer is fractional, split.** The relaxed answer says DNR-003 is 0.4 assigned to REQ-C. So consider two worlds: one where DNR-003 definitely goes to REQ-C, one where they definitely do not. Solve both.
3. **Discard whole branches without exploring them.** The relaxed answer is always at least as good as any real answer beneath it. So if a branch's relaxed score is already worse than the best real solution found so far, every arrangement in that branch is worse too, and the entire branch can be thrown away unexamined.
4. **Repeat** until every branch is either solved or discarded.

Step 3 is what makes it work. Most of the possibilities are eliminated in blocks rather than one at a time.

Why it is unusually fast on this particular problem. Assignment problems have a special mathematical structure. When you relax the yes-or-no rule, the answer usually comes back as whole numbers anyway, without being forced. Little branching is needed. This is why the solver proved optimality on a problem with 72,000 yes-or-no decisions in under four seconds, while the evolutionary algorithm took over two minutes to produce something merely close.

Tracing the whole set of trade-offs

One solve gives one answer, optimal for one goal. To produce the full menu of trade-offs, the solver is run repeatedly with limits imposed on the other goals: "find the best fulfilment, given that you waste no more than 4 units of universal blood, and travel no more than 15 km." Sweeping those limits traces out the range of sensible compromises. On the Tuesday morning example this took 220 solves and produced 46 distinct trade-offs.

9. What was measured

DONORS	YES-OR-NO DECISIONS	EXACT SOLVER	EVOLUTIONARY ALGORITHM
40	80	0.07 s	4.05 s
320	5,120	0.18 s	9.74 s
640	20,480	1.30 s	38.2 s
1,200	72,000	3.68 s, proven optimal	139.4 s, approximate

The exact method was faster at every size tested, and the gap widened rather than closed. It also returns a guarantee: this is the best arrangement, not merely a good one. And it returns the same answer every time, which matters when somebody asks why a particular donor was chosen.

The honest conclusion. At the scale a regional blood service actually operates at, this problem can be solved exactly. Claiming that it needs an evolutionary algorithm would be overstating the difficulty.

10. When this stops working

The exact method depends entirely on everything staying linear. Three realistic developments would break that, and each is where NSGA-III earns its place:

- **National scale.** Tens of thousands of donors and hundreds of simultaneous requests. The measurements above found no crossover, but one exists somewhere.
- **Donor responses that are not independent.** The logarithm trick assumes each donor decides on their own. If a public holiday or a transport strike makes a whole area unavailable at once, the product no longer separates and the linear form is lost.
- **Planning across weeks rather than one morning.** Using a donor today makes them ineligible for ninety days. Optimising over a horizon rather than a moment is a much larger and less tractable model.

This is why the project keeps both methods. The exact solver is the working method and the source of ground truth; NSGA-III is measured against it, so that when the problem does grow beyond exact solving,

there is evidence for how much accuracy is given up.

All donor names, response rates and distances in this document are fictional demonstration data from the BloodGrid prototype. No real donor, patient, or blood-bank data was used. The optimisation results were computed by the implementation in ccp1/ and can be reproduced with `python -m ccp1.run_experiment compare`.